Interpretability-Guided Training for Robust Jet Tagging Under Simulation-to-Data Shift

Preliminary Research

Ryan Yang

1 Progress Made and Difficulties Encountered

Since the project proposal I have made good progress on the JetClass side of the project. I now have a working baseline Particle Transformer training pipeline on JetClass, a post-training corruption study, an interpretability benchmarking pipeline, and an initial implementation of the interpretability-guided training in the style of input gradient regularization [3].

The current completed pieces are:

- a baseline Particle Transformer trained on JetClass constituent-level features;
- a corruption-based benchmark on JetClass test data, used to study how unlabeled shift metrics correlate with supervised degradation;
- an interpretability benchmark comparing input gradients, Integrated Gradients, and SmoothGrad using targeted-versus-random masking tests;
- Slurm batch scripts and analysis code to make these studies reproducible across random seeds.

The main technical difficult so far has been the benchmarking. AspenOpenJets [1] is an unlabeled real particle data dataset, so it doesn't support a direct supervised test accuracy or AUC in the same way that JetClass [2] does. This made it so I had to refine the project setup: instead of trying to claim real-data accuracy directly, I now calibrate the unlabeled shift diagnostics on controlled corruptions of JetClass, where the true labels are known, and then plan to apply the same diagnostics to AspenOpenJets later.

A second difficult I encountered was the trainer-native metrics and our custom post-training inference path were not always numerically consistent. In particular, my custom posthoc inference runs produced unrealistically poor top-1 accuracy even when the trainer itself showed strong validation performance. To address this, I changed the metrics to use exactly trainer-native style analysis. This was an important practical sanity step to keep metrics reliable and strong.

Finally, I also implemented an intial interpretability-guided training code path using gradient regularization, but I do not yet have the clean, stable results from that stage. In other words, the code infrastructure for the next phase is in place, but the results are still in progress.

2 Refined Scientific Question

The original proposal asked whether interpretability-guided training can make transformer-based jet taggers rely on more stable and physically meaningful features under the simulation-to-data shift. That stays as the core idea, but I have sharpened the question to make the benchmarking strategy more concrete:

> *Can interpretability-guided training improve the robustness of a Particle Transformer jet classifier under simulation-to-data shift while preserving strong supervised performance on JetClass, and can that robustness be quantified on unlabeled target-domain data using shift metrics that are first validated on controlled JetClass corruptions?*

This refinement is important for two reasons. First of all, it makes the baseline mode explicit: it's a Particle Transformer trained on JetClass constituent-level features. Second, it replaces the vague idea of compare JetClass to AspenOpenJets with a systematic two-step benchmark:

- measure ordinary supervised performance on JetClass
- validate unlabeled shift metrics on corrupted JetClass before using them on AspenOpenJets

The currently implemented candidate shift metrics are (might end up trying more, might not):

- prediction entropy shift;
- confidence drop;
- top-1 flip rate under perturbation;
- mean probability drift $\|\hat{\mathbf{p}}(x) - \hat{\mathbf{p}}(T(x))\|_1$;
- class-distribution shift measured with Jensen–Shannon divergence.

3 Key Equations

The baseline classifier is trained with multi-class cross-entropy:

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log \hat{p}_{ic}, \tag{1}$$

where y_{ic} is the one-hot truth label for jet i and class c, and $\hat{p}_{ic}$ is the predicted probability.

The supervised test accuracy is

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\left[\arg\max_c \hat{p}_{ic} = \arg\max_c y_{ic}\right]. \tag{2}$$

One important uncertainty-based diagnostic is predictive entropy:

$$H(x) = -\sum_{c=1}^{C}\hat{p}_c(x)\log \hat{p}_c(x). \tag{3}$$

To quantify prediction stability under a perturbation T, I use the mean probability drift

$$S_{\mathrm{pert}} = \frac{1}{N}\sum_{i=1}^{N}\|\hat{\mathbf{p}}(x_i) - \hat{\mathbf{p}}(T(x_i))\|_1, \tag{4}$$

and also the top-1 flip rate

$$\mathrm{FlipRate} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\left[\arg\max_c \hat{p}_c(x_i) \neq \arg\max_c \hat{p}_c(T(x_i))\right]. \tag{5}$$

To compare class distributions between two datasets A and B, I use the average predicted class vectors

$$\bar{\mathbf{p}}_A = \frac{1}{N_A}\sum_{i\in A}\hat{\mathbf{p}}(x_i), \qquad \bar{\mathbf{p}}_B = \frac{1}{N_B}\sum_{i\in B}\hat{\mathbf{p}}(x_i), \tag{6}$$

and a divergence such as

$$D_{\mathrm{JS}}(\bar{\mathbf{p}}_A, \bar{\mathbf{p}}_B). \tag{7}$$

To validate whether an unlabeled shift metric M is useful, I measure its correlation with supervised degradation on corrupted JetClass:

$$\rho_{\mathrm{S}}(M, \Delta\mathrm{AUC}) \qquad \text{and} \qquad \rho_{\mathrm{S}}(M, \Delta\mathrm{Acc}). \tag{8}$$

For interpretability-guided training, the main equation currently implemented is an RRR-style gradient penalty:

$$\mathcal{L}_{\mathrm{RRR}} = \mathcal{L}_{\mathrm{CE}} + \lambda \left\| A \odot \nabla_x s_\theta(x) \right\|_2^2, \tag{9}$$

where $s_\theta(x)$ is a chosen model score, λ is the regularization strength, and A is either an all-feature mask or an adaptive mask that emphasizes the currently most influential feature channels [3].

For attribution itself, one method used in the project is Integrated Gradients [4]:

$$\mathrm{IG}_j(x) = (x_j - x'_j) \int_0^1 \frac{\partial s(x' + \alpha(x - x'))}{\partial x_j} \, d\alpha. \tag{10}$$

4 Code Used to Answer the Question

The working repository for this project is:

`https://github.com/ryanlyang/CompPhys_Final`

Most of the base code structure is inherited from:

`https://github.com/jet-universe/particle_transformer` [2]

This inherited portion includes the underlying Particle Transformer model/network setup, dataset configuration files, and core Weaver-based training infrastructure.

The project-specific code that I added or substantially modified is:

- `run_jetclass_part0_baseline_and_shift.py`: baseline evaluation and corruption-based shift benchmark;
- `run_jetclass_interpretability_benchmark.py`: attribution benchmarking with targeted-versus-random masking;
- `train_jetclass_part0_rrr.py`: interpretability-guided training via input-gradient regularization;
- custom Slurm scripts under `sbatch/` for reproducible multi-seed runs, eval-only runs, and dependency-based scheduling on the research cluster;
- reporting and analysis integration updates (for example, trainer-metric based clean-performance reporting when posthoc inference is unstable).

The current output products include trainer summaries, corruption metric tables, correlation tables, interpretability summaries, and method-comparison CSV files.

5 Preliminary Results

All results in this section are from runs with **150,000 training jets, 50,000 validation jets, and 150,000 test jets per seed**. Interpretability runs use **120,000 evaluation jets** and a **20,000-jet stratified explanation subset** (2,000 jets per class).

Unless otherwise stated, aggregate values are reported as **mean $\pm$ standard deviation over 3 randomization seeds** ($n = 3$).

Values were copied directly from:

- `summary.json` (clean metrics and diagnostics),
- `corruption_metrics.csv` and `correlations.csv`,
- `interpretability_summary.json` and `method_effectiveness_summary.csv`.

How to read these tables

To make interpretation explicit, the main quantities are defined as follows:

- **Best val metric**: best validation classification metric reported by the trainer log (used as the trainer-native reference).
- **Test acc**: top-1 test accuracy, i.e. fraction of jets with correct predicted class.
- **Macro AUC**: one-vs-rest ROC AUC averaged equally across the 10 classes.
- **Mean entropy**: average predictive entropy $-\sum_c \hat{p}_c \log \hat{p}_c$; larger means less certainty.
- **Mean confidence**: average maximum class probability $\max_c \hat{p}_c$.
- **ΔAcc, ΔAUC**: clean metric minus corrupted metric. Positive values mean performance degradation under corruption.
- **ΔEntropy**: corrupted entropy minus clean entropy. Positive values mean the model became more uncertain.
- **Conf. drop**: clean confidence minus corrupted confidence. Positive values mean confidence decreased under corruption.
- **Flip rate**: fraction of jets whose top-1 predicted class changes between clean and corrupted inputs.
- **JS div**: Jensen–Shannon divergence between clean and corrupted predicted class distributions.
- **Spearman/Pearson correlations**: association between each unlabeled shift metric and supervised degradation across corruption levels.
- **Targeted drop vs random drop**: in interpretability tests, targeted masking removes high-attribution constituents while random masking removes the same fraction uniformly at random. The primary "drop" here is the decrease in target-class probability, $p_{c^*}(\text{clean}) - p_{c^*}(\text{masked})$, averaged over jets and mask fractions; AUC-drop and accuracy-drop gaps are also reported as secondary checks.
- **Gap (target-rand)**: targeted minus random effect. A larger positive gap indicates the method better identifies constituents that are actually important for model output.

For this report, trainer-native metrics are treated as authoritative for clean supervised performance, and posthoc values are used as diagnostics when needed.

Table 1: Clean performance across seeds (trainer-native)

Seed	Best val metric	Test acc	Macro AUC	Mean entropy	Mean confidence
0	0.75317	0.74597	0.60437	1.65931	0.36581
1	0.74977	0.74899	0.63343	1.73336	0.33616
2	0.75222	0.75018	0.65396	1.78058	0.32593
Mean $\pm$ Std	0.75172 ± 0.00175	0.74838 ± 0.00217	0.63059 ± 0.02492	1.72442 ± 0.06113	0.34263 ± 0.02071

Table 1: Primary clean-performance table. Trainer-native metrics are treated as authoritative.

Table 2: Top shift-metric ranking by seed (Spearman with ΔAUC)

Table 3: Calibration of unlabeled shift metrics

Table 4: Interpretability method effectiveness

Table 5: Sanity checks and outcomes

Quick Discussion of Preliminary Results

Table 1 shows stable clean performance across three seeds, with trainer-native accuracy 0.74838 ± 0.00217 and macro AUC 0.63059 ± 0.02492. Table 3 indicates that class-JS divergence, top-1 flip rate, and probability L1 drift are the

Seed	Rank 1	Rank 2	Rank 3
0	entropy_shift (0.952)	confidence_drop (0.952)	class_js_div (0.952)
1	class_js_div (0.752)	prob_l1_drift (0.700)	top1_flip_rate (0.697)
2	class_js_div (0.967)	entropy_shift (0.887)	top1_flip_rate (0.725)

Table 2: Top three unlabeled shift metrics ranked by absolute Spearman correlation with AUC degradation, shown per seed.

Metric	Spearman(ΔAUC)	Pearson(ΔAUC)	Spearman(ΔAcc)	Pearson(ΔAcc)
Prob. L1 drift	0.699 ± 0.006	0.993 ± 0.012	0.693 ± 0.012	0.985 ± 0.014
Top-1 flip rate	0.706 ± 0.016	0.993 ± 0.010	0.699 ± 0.023	0.986 ± 0.014
Class JS divergence	0.890 ± 0.120	0.987 ± 0.010	0.872 ± 0.112	0.977 ± 0.027
Confidence drop	0.468 ± 0.600	0.324 ± 1.116	0.457 ± 0.574	0.348 ± 1.094
Entropy shift	0.624 ± 0.513	0.427 ± 0.957	0.626 ± 0.470	0.470 ± 0.897

Table 3: Calibration table: which unlabeled metric best tracks supervised degradation on corrupted JetClass.

most useful unlabeled metrics for tracking supervised degradation under controlled corruptions. Table 4 shows that, under the current masking setup, input gradients, Integrated Gradients, and SmoothGrad give very similar targeted-versus-random gaps, so no single attribution method clearly outperforms the others yet.

6 Sanity Checks and GIGO Prevention

I have performed several sanity checks to reduce the chance of conclusions drawn from a broken or misleading codebase:

- **Class-balance checks.** The analysis scripts explicitly print and record the class distribution in each split. For example, the JetClass test split is checked to confirm equal representation across the 10 classes rather than an accidental class imbalance.
- **Disjoint split construction.** The train, validation, and test sets are built from disjoint file-index ranges, which reduces the risk of train–test leakage.
- **Label-mapping sanity check.** I compute a permutation diagnostic comparing raw accuracy to the best label permutation accuracy. This is useful for detecting class-index mismatches or label-order bugs.
- **Numerical stability checks.** The posthoc analysis code explicitly checks for non-finite logits and probabilities. This turned out to be essential, because some custom inference paths produced unstable outputs even when the trainer-native metrics were reasonable.
- **Trainer-versus-posthoc consistency check.** Clean supervised performance is compared against trainer-native metrics from logs or summaries. If the custom posthoc path is inconsistent, I now report the trainer-native metric as authoritative and treat the posthoc result as diagnostic only.
- **Attribution sanity check.** For interpretability benchmarking, targeted masking and random masking are performed at the same masking fractions, so differences are not simply caused by removing more constituents in one condition than the other.
- **Multi-seed reproducibility.** The pipeline is designed to run across multiple random seeds, so the final project can report whether a result is robust or only appears in a single lucky run.

These sanity checks have been more than just formalities, they have caught several issues in the pipeline, especially the mismatch between the trainer-native and posthoc metrics. That debugging work is part of the scientific progress of the project because it ensures that future conclusions are based on reliable outputs rather than garbage-in, garbage-out.

Method	Targeted drop	Random drop	Gap (target-rand)	AUC gap	Acc gap
Input gradients	-0.0610 ± 0.0112	-0.0413 ± 0.0101	-0.0197 ± 0.0064	-0.0264 ± 0.0081	-0.0179 ± 0.0060
Integrated Gradients	-0.0742 ± 0.0104	-0.0401 ± 0.0092	-0.0341 ± 0.0053	-0.0412 ± 0.0070	-0.0287 ± 0.0051
SmoothGrad	-0.0668 ± 0.0121	-0.0424 ± 0.0103	-0.0244 ± 0.0071	-0.0315 ± 0.0092	-0.0208 ± 0.0068

Table 4: Method ranking from targeted-versus-random masking (mean $\pm$ std over 3 seeds).

Check	Observed quantity	Expected behavior	Status
Class balance in eval split	15,000 jets/class (10 classes; 150,000 total)	Approximately balanced by construction	Pass
Permutation diagnostic	$\Delta_{\mathrm{perm}} = 0.00404 \pm 0.00160$	Small Δ when label mapping is correct	Pass
Targeted vs random masking parity	Same mask fractions in both settings (0.02, 0.05, 0.10, 0.20)	Fair comparison by design	Pass
Non-finite output checks	Non-finite checks enabled; trainer-native metrics used for reported clean accuracy	Near-zero after stable eval path	Pass (with safeguards)

Table 5: Sanity checks used to prevent garbage-in, garbage-out conclusions.

7 Current Status

At the time of this preliminary report, the baseline JetClass study is close to complete: the baseline training, corruption-based benchmarking, and the interpretability benchmarking are all in place and running. The AspenOpenJets stage is still in progress. Likewise, the interpretability-guided training code is implmeented, but not yet stable enough or debugged enough to present results from that stage.

This means the project has made decent progress: the baseline system is in place, the benchmarking strategy has been made concrete, multiple diagnostics and sanity checks are implmeented, and the next tasks are AspenOpenJet studies and stable interpretability-guided training.

References

[1] Oz Amram, Luca Anzalone, Joschka Birk, Darius A Faroughy, Anna Hallin, Gregor Kasieczka, Michael Krämer, Ian Pang, Humberto Reyes-Gonzalez, and David Shih. Aspen open jets: unlocking lhc data for foundation models in particle physics. *Machine Learning: Science and Technology*, 6(3):030601, 2025.

[2] Huilin Qu, Congqiao Li, and Sitian Qian. Particle transformer for jet tagging. In *International Conference on Machine Learning*, pages 18281–18292. PMLR, 2022.

[3] Andrew Slavin Ross, Michael C Hughes, and Finale Doshi-Velez. Right for the right reasons: Training differentiable models by constraining their explanations. *arXiv preprint arXiv:1703.03717*, 2017.

[4] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.